

```

    public void handle(String message);
    public String getType();
}

```

The refactored implementation of the broker maintains the list of handlers.

```

package com.glarimy.framework;

import java.util.Date;
import java.util.HashMap;
import java.util.Map;

public class MessageBroker implements Broker {
    private Map<Long, Handler> handlers;

    public MessageBroker() {
        handlers = new HashMap<Long, Handler>();
    }
    ...
}

```

And the registration and removal of the handlers are on the expected lines.

```

@Override
public long register(Handler handler) {
    long hid = new Date().getTime();
    handlers.put(hid, handler);
    return hid;
}

@Override
public void unregister(long sid) {
    handlers.remove(sid);
}

```

However, the *publish()* method requires small modifications. Earlier, all the messages were delivered to all the subscribers. That has to be changed. It must only deliver messages to the handlers that are interested in the type of the message. Accordingly, the following is the updated *publish()* method.

```

@Override
public void publish(String message, String type) {
    handlers.values().stream()
        .filter(h -> h.getType().equalsIgnoreCase(type))
        .forEach(s -> s.handle(message));
}

```

Figure 2 depicts the updated broker design. We will revisit the design of the message broker

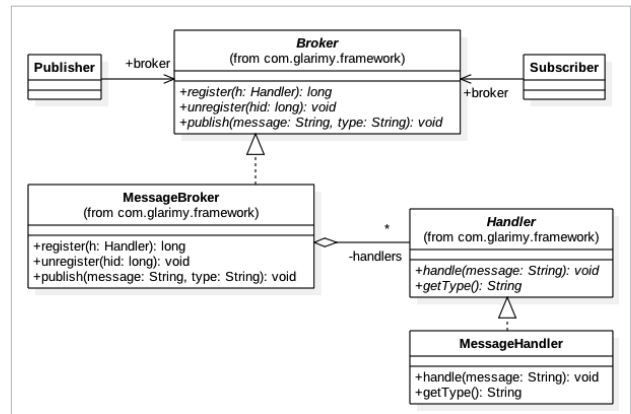


Figure 2: Messaging infrastructure with handlers

in the future for a deeper treatment. Since our current focus is on the fundamental object-oriented patterns, the *MessageBroker* in the current form is just enough to demonstrate the behavioural patterns.

UMS with messaging

The existing user management system (UMS) developed so far is intrinsically synchronous in nature. The consumers and providers are aware of each other. With the *MessageBroker* at its disposal, the UMS can be upgraded to use asynchronous communication.

Instead of the *UserRepositoryJournalProxy* directly calling the *JournalAdapter*, we can decouple them by using the broker. Let the *UserRepositoryJournalProxy* send messages to the broker and thereby the *JournalHandler* processes the messages. To connect these two virtually, the messages can be typed as *com.glarimy.ums*.

Following is the implementation of *JournalHandler*:

```

package com.glarimy.ums;

import com.glarimy.framework.Factory;
import com.glarimy.framework.Handler;
import com.glarimy.framework.ObjectFactory;

public class JournalHandler implements Handler {
    private JournalAdapter adapter;

    public JournalHandler() throws Exception {
        Factory factory = new ObjectFactory("config.
properties");
        this.adapter = (JournalAdapter) factory.
get("adapter");
    }

    @Override
    public void handle(String message) {

```